

# Learning Arithmetic Formulas in the Presence of Noise

## A General Framework and Applications to Unsupervised Learning

Pritam Chandra, Ankit Garg, Neeraj Kayal, S  
Kunal Mittal, Tanmay Sinha

Feb. 22, 2024

# Summary

- 1 Robust Vector Space Decomposition (RVSD)
- 2 Learning Arithmetic Formulas
- 3 Applications: Unsupervised Learning

# Robust Vector Space Decomposition (RVSD)

**Setting:** Let  $U$  and  $V$  be subspaces of some ambient space. Let  $\mathcal{B}$  be a (finite) collection of operators in  $\text{Lin}(U, V)$  such that it respects a certain direct sum decomposition of  $U$  and  $V$ . That is

$$U = U_1 \oplus \cdots \oplus U_s$$

$$V = V_1 \oplus \cdots \oplus V_s$$

$$BU_j \subseteq V_j \quad \forall j, \quad \forall B \in \mathcal{B}.$$

**Setting:** Let  $U$  and  $V$  be subspaces of some ambient space. Let  $\mathcal{B}$  be a (finite) collection of operators in  $\text{Lin}(U, V)$  such that it respects a certain direct sum decomposition of  $U$  and  $V$ . That is

$$U = U_1 \oplus \cdots \oplus U_s$$

$$V = V_1 \oplus \cdots \oplus V_s$$

$$BU_j \subseteq V_j \quad \forall j, \quad \forall B \in \mathcal{B}.$$

**Problem:** Given  $U, V, \mathcal{B}$ , find the  $U_j$ s (and  $V_j$ s) up to re-ordering.

**Setting:** Let  $U$  and  $V$  be subspaces of some ambient space. Let  $\mathcal{B}$  be a (finite) collection of operators in  $\text{Lin}(U, V)$  such that it respects a certain direct sum decomposition of  $U$  and  $V$ . That is

$$\begin{aligned} U &= U_1 \oplus \cdots \oplus U_s \\ V &= V_1 \oplus \cdots \oplus V_s \\ BU_j &\subseteq V_j \quad \forall j, \quad \forall B \in \mathcal{B}. \end{aligned}$$

**Problem:** Given  $U, V, \mathcal{B}$ , find the  $U_j$ s (and  $V_j$ s) up to re-ordering.

**Robust version:** Given  $\tilde{U}, \tilde{V}, \tilde{\mathcal{B}}$  close to  $U, V, \mathcal{B}$ , find the  $\tilde{U}_j$ s close to  $U_j$ s up to re-ordering.

# Intuition

Let  $\mathcal{B} = (B_1, \dots, B_m)$ .

Consider the matrix  $M$  whose columns are a basis of  $U$  that respects the above decomposition. That is the first few columns are a basis for  $U_1$ , the next few columns for  $U_2$ , and so on. Similarly, consider  $N$  for  $V$ .

# Intuition

Let  $\mathcal{B} = (B_1, \dots, B_m)$ .

Consider the matrix  $M$  whose columns are a basis of  $U$  that respects the above decomposition. That is the first few columns are a basis for  $U_1$ , the next few columns for  $U_2$ , and so on. Similarly, consider  $N$  for  $V$ .

**Observation:** Each matrix  $B_i$  is block diagonal in these bases. That is

$$B_i = N \begin{bmatrix} B_{i1} & & \\ & \ddots & \\ & & B_{is} \end{bmatrix} M^{-1}.$$



Consider  $P_j$ , the projection onto  $U_j$  along

$$U_1 \oplus \cdots \oplus U_{j-1} \oplus U_{j+1} \oplus \cdots \oplus U_s.$$

Similarly, consider  $\Pi_j$  for the projection onto  $V_j$ . Then we have

Consider  $P_j$ , the projection onto  $U_j$  along

$$U_1 \oplus \cdots \oplus U_{j-1} \oplus U_{j+1} \oplus \cdots \oplus U_s.$$

Similarly, consider  $\Pi_j$  for the projection onto  $V_j$ . Then we have

$$B_i = N \begin{bmatrix} B_{i1} & & \\ & \ddots & \\ & & B_{is} \end{bmatrix} M^{-1}, \quad P_j = M \begin{bmatrix} 0 & & \\ & I & \\ & & 0 \end{bmatrix} M^{-1} \text{ and } \Pi_j = N \begin{bmatrix} 0 & & \\ & I & \\ & & 0 \end{bmatrix} N^{-1}$$

(here the  $I$  is in the  $j$ -th diagonal block, and is of appropriate dimension in each case). Then

Consider  $P_j$ , the projection onto  $U_j$  along

$$U_1 \oplus \cdots \oplus U_{j-1} \oplus U_{j+1} \oplus \cdots \oplus U_s.$$

Similarly, consider  $\Pi_j$  for the projection onto  $V_j$ . Then we have

$$B_i = N \begin{bmatrix} B_{i1} & & \\ & \ddots & \\ & & B_{is} \end{bmatrix} M^{-1}, \quad P_j = M \begin{bmatrix} 0 & & \\ & I & \\ & & 0 \end{bmatrix} M^{-1} \text{ and } \Pi_j = N \begin{bmatrix} 0 & & \\ & I & \\ & & 0 \end{bmatrix} N^{-1}$$

(here the  $I$  is in the  $j$ -th diagonal block, and is of appropriate dimension in each case). Then

$$B_i P_j = \Pi_j B_i = N \begin{bmatrix} 0 & & \\ & B_{ij} & \\ & & 0 \end{bmatrix} M^{-1}, \quad \forall i, j.$$

Note that  $U_j$  is the column space of  $P_j$ . So finding  $P_j$  is sufficient.

Note that  $U_j$  is the column space of  $P_j$ . So finding  $P_j$  is sufficient.

As  $B_i P_j - \Pi_j B_i = 0$ , finding  $P_j$  connects to finding matrices  $D$  and  $E$  such that

$$B_i D - E B_i = 0 \quad \forall i.$$

Note that  $U_j$  is the column space of  $P_j$ . So finding  $P_j$  is sufficient.

As  $B_i P_j - \Pi_j B_i = 0$ , finding  $P_j$  connects to finding matrices  $D$  and  $E$  such that

$$B_i D - E B_i = 0 \quad \forall i.$$

This leads to the notion of the *adjoint algebra*.

## Adjoint Algebra of $\mathcal{B}$

$$\text{Adj}(\mathcal{B}) = \{(D, E) \in \text{Lin}(U, U) \times \text{Lin}(V, V) \mid B_i D - E B_i = 0, \forall i\}$$

## Adjoint Algebra of $\mathcal{B}$

$$\text{Adj}(\mathcal{B}) = \{(D, E) \in \text{Lin}(U, U) \times \text{Lin}(V, V) \mid B_i D - E B_i = 0, \forall i\}$$

Facts:

- $\text{Adj}$  is an algebra.



## Adjoint Algebra of $\mathcal{B}$

$$\text{Adj}(\mathcal{B}) = \{(D, E) \in \text{Lin}(U, U) \times \text{Lin}(V, V) \mid B_i D - E B_i = 0, \forall i\}$$

Facts:

- $\text{Adj}$  is an algebra.
- We saw earlier that  $(P_j, \Pi_j) \in \text{Adj}$  for all  $j$ .

## Adjoint Algebra of $\mathcal{B}$

$$\text{Adj}(\mathcal{B}) = \{(D, E) \in \text{Lin}(U, U) \times \text{Lin}(V, V) \mid B_i D - E B_i = 0, \forall i\}$$

Facts:

- Adj is an algebra.
- We saw earlier that  $(P_j, \Pi_j) \in \text{Adj}$  for all  $j$ .
- We say that the adjoint algebra is **non-degenerate** if  $\dim \text{Adj} = s$ .  
That is

$$\text{Adj} = \text{span} \{(P_1, \Pi_1), \dots, (P_s, \Pi_s)\}.$$

... How to solve the problem (without noise)?

## Adjoint Algebra Operator

$\mathfrak{A}_{\mathcal{B}} : \text{Lin}(U, U) \times \text{Lin}(V, V) \rightarrow \text{Lin}(U, V)^m$  defined as

$$\mathfrak{A}(D, E) = (B_1 D - E B_1, \dots, B_m D - E B_m) .$$

## Adjoint Algebra Operator

$\mathfrak{A}_{\mathcal{B}} : \text{Lin}(U, U) \times \text{Lin}(V, V) \rightarrow \text{Lin}(U, V)^m$  defined as

$$\mathfrak{A}(D, E) = (B_1 D - E B_1, \dots, B_m D - E B_m) .$$

$\text{Adj}(\mathcal{B})$  is the null-space of  $\mathfrak{A}_{\mathcal{B}}$ .

Let  $(P, \Pi)$  be a random element of a non-degenerate Adj. Then we know that

$$P = \lambda_1 P_1 + \cdots + \lambda_s P_s.$$

Let  $(P, \Pi)$  be a random element of a non-degenerate Adj. Then we know that

$$P = \lambda_1 P_1 + \cdots + \lambda_s P_s.$$

... How to find the  $P_j$ s from  $P$ ?

Let  $(P, \Pi)$  be a random element of a non-degenerate Adj. Then we know that

$$P = \lambda_1 P_1 + \cdots + \lambda_s P_s.$$

... How to find the  $P_j$ s from  $P$ ?

- Look at the map  $\hat{P} : \text{Adj} \rightarrow \text{Adj}$  such that  $\hat{P}(X) = PX$ .



Let  $(P, \Pi)$  be a random element of a non-degenerate Adj. Then we know that

$$P = \lambda_1 P_1 + \cdots + \lambda_s P_s.$$

... How to find the  $P_j$ s from  $P$ ?

- Look at the map  $\hat{P} : \text{Adj} \rightarrow \text{Adj}$  such that  $\hat{P}(X) = PX$ .
- As  $\dim \text{Adj} = s$ , the map  $\hat{P}$  has at most  $s$  eigenvectors.

Let  $(P, \Pi)$  be a random element of a non-degenerate Adj. Then we know that

$$P = \lambda_1 P_1 + \cdots + \lambda_s P_s.$$

... How to find the  $P_j$ s from  $P$ ?

- Look at the map  $\hat{P} : \text{Adj} \rightarrow \text{Adj}$  such that  $\hat{P}(X) = PX$ .
- As  $\dim \text{Adj} = s$ , the map  $\hat{P}$  has at most  $s$  eigenvectors.
- As  $P_j$ s are disjoint projections, we have  $\hat{P}(P_j) = \lambda_j P_j$ .

Let  $(P, \Pi)$  be a random element of a non-degenerate Adj. Then we know that

$$P = \lambda_1 P_1 + \cdots + \lambda_s P_s.$$

... How to find the  $P_j$ s from  $P$ ?

- Look at the map  $\hat{P} : \text{Adj} \rightarrow \text{Adj}$  such that  $\hat{P}(X) = PX$ .
- As  $\dim \text{Adj} = s$ , the map  $\hat{P}$  has at most  $s$  eigenvectors.
- As  $P_j$ s are disjoint projections, we have  $\hat{P}(P_j) = \lambda_j P_j$ .
- Therefore,  $P_j$ s are exactly the eigenvectors of  $\hat{P}$ .

## Problem (noiseless)

Given  $U, V$  and  $\mathcal{B} = \{B_1, \dots, B_m\}$  such that

$$U = U_1 \oplus \dots \oplus U_s$$

$$V = V_1 \oplus \dots \oplus V_s$$

$$B_i U_j \subseteq V_j \quad \forall i, j$$

find the  $U_i$ s.

## Algorithm

1. Construct the map  $\mathfrak{A}_{\mathcal{B}}$ .
2. Pick a random element  $(P, \Pi)$  from its null space  $\text{Adj}(\mathcal{B})$ .
3. Construct the map  $\hat{P}$ .
4. Output the column spaces of the the eigenvectors of  $\hat{P}$ .

... Solving the robust version.

# Eigenvector Perturbation

## Theorem

Let  $A = X\Lambda X^{-1}$  be the eigendecomposition of  $A$ . Suppose  $A$  has distinct eigenvalues with a minimum eigenvalue gap of  $\delta > 0$ . Let  $\tilde{A} = A + E$ . Further, let  $\kappa(X) \|E\| \geq \frac{\delta}{2}$ . Then for each eigenpair  $(\lambda, x)$  of  $A$  there is an eigenpair  $(\tilde{\lambda}, \tilde{x})$  such that

- $|\lambda - \tilde{\lambda}| < \kappa(X) \|E\|,$
- $\|x - \tilde{x}\| \leq \frac{4\kappa(X)\|E\|}{\delta}.$

# Null Space Perturbation

## Lemma

*(Lemma G.5 in [1]) Suppose  $A$  has rank  $r$ . And let  $\tilde{A} = A + E$ . Let  $S, \tilde{S}$  be the subspaces spanned by the top  $r$  right singular vectors of  $A$  and  $\tilde{A}$  respectively. Then we have:*

$$\text{dist}(S, \tilde{S}) \leq \frac{2 \|E\|}{\sigma_r(A)}.$$

# Algorithm

1. Construct the map  $\mathfrak{A}_{\tilde{B}}$ .
2. Pick a random element  $(\tilde{P}, \tilde{\Pi})$  from the space  $\widetilde{\text{Adj}(B)}$  spanned by the  $s$  smallest singular vectors of  $\mathfrak{A}_{\tilde{B}}$ .
3. Construct the map  $\hat{\tilde{P}}$ . Here  $\hat{\tilde{P}}(X) = \text{Proj}_{\widetilde{\text{Adj}(B)}}(\tilde{P}X)$ .
4. Output the span of the top singular vectors of  $\hat{\tilde{P}}$ .



# Algorithm

1. Construct the map  $\mathfrak{A}_{\tilde{\mathcal{B}}}$ .
2. Pick a random element  $(\tilde{P}, \tilde{\Pi})$  from the space  $\widetilde{\text{Adj}(\mathcal{B})}$  spanned by the  $s$  smallest singular vectors of  $\mathfrak{A}_{\tilde{\mathcal{B}}}$ .
3. Construct the map  $\hat{\tilde{P}}$ . Here  $\hat{\tilde{P}}(X) = \text{Proj}_{\widetilde{\text{Adj}(\mathcal{B})}}(\tilde{P}X)$ .
4. Output the span of the top singular vectors of  $\hat{\tilde{P}}$ .

1.  $\|\mathfrak{A}_{\mathcal{B}} - \mathfrak{A}_{\tilde{\mathcal{B}}}\|$  depends on  $\|\mathcal{B} - \tilde{\mathcal{B}}\|$ .

# Algorithm

1. Construct the map  $\mathfrak{A}_{\tilde{\mathcal{B}}}$ .
2. Pick a random element  $(\tilde{P}, \tilde{\Pi})$  from the space  $\widetilde{\text{Adj}(\mathcal{B})}$  spanned by the  $s$  smallest singular vectors of  $\mathfrak{A}_{\tilde{\mathcal{B}}}$ .
3. Construct the map  $\hat{\tilde{P}}$ . Here  $\hat{\tilde{P}}(X) = \text{Proj}_{\widetilde{\text{Adj}(\mathcal{B})}}(\tilde{P}X)$ .
4. Output the span of the top singular vectors of  $\hat{\tilde{P}}$ .

1.  $\|\mathfrak{A}_{\mathcal{B}} - \mathfrak{A}_{\tilde{\mathcal{B}}}\|$  depends on  $\|\mathcal{B} - \tilde{\mathcal{B}}\|$ .
2.  $\|P - \tilde{P}\|_F \leq \text{dist}\left(\text{Adj}(\mathcal{B}), \widetilde{\text{Adj}(\mathcal{B})}\right) \leq \mathcal{O}\left(\|\mathfrak{A}_{\mathcal{B}} - \mathfrak{A}_{\tilde{\mathcal{B}}}\|, \frac{1}{\sigma_{-(s+1)}(\mathfrak{A}_{\mathcal{B}})}\right)$

# Algorithm

1. Construct the map  $\mathfrak{A}_{\tilde{\mathcal{B}}}$ .
2. Pick a random element  $(\tilde{P}, \tilde{\Pi})$  from the space  $\widetilde{\text{Adj}(\mathcal{B})}$  spanned by the  $s$  smallest singular vectors of  $\mathfrak{A}_{\tilde{\mathcal{B}}}$ .
3. Construct the map  $\hat{\tilde{P}}$ . Here  $\hat{\tilde{P}}(X) = \text{Proj}_{\widetilde{\text{Adj}(\mathcal{B})}}(\tilde{P}X)$ .
4. Output the span of the top singular vectors of  $\hat{\tilde{P}}$ .

1.  $\|\mathfrak{A}_{\mathcal{B}} - \mathfrak{A}_{\tilde{\mathcal{B}}}\|$  depends on  $\|\mathcal{B} - \tilde{\mathcal{B}}\|$ .
2.  $\|P - \tilde{P}\|_F \leq \text{dist}\left(\text{Adj}(\mathcal{B}), \widetilde{\text{Adj}(\mathcal{B})}\right) \leq \mathcal{O}\left(\|\mathfrak{A}_{\mathcal{B}} - \mathfrak{A}_{\tilde{\mathcal{B}}}\|, \frac{1}{\sigma_{-(s+1)}(\mathfrak{A}_{\mathcal{B}})}\right)$
3.  $\|P_j - \tilde{P}_j\|_F \leq \mathcal{O}\left(\|P - \tilde{P}\|_F, \kappa(M), \frac{1}{\text{eigengap}(P)}\right).$

# Algorithm

1. Construct the map  $\mathfrak{A}_{\tilde{\mathcal{B}}}$ .
2. Pick a random element  $(\tilde{P}, \tilde{\Pi})$  from the space  $\widetilde{\text{Adj}(\mathcal{B})}$  spanned by the  $s$  smallest singular vectors of  $\mathfrak{A}_{\tilde{\mathcal{B}}}$ .
3. Construct the map  $\hat{\tilde{P}}$ . Here  $\hat{\tilde{P}}(X) = \text{Proj}_{\widetilde{\text{Adj}(\mathcal{B})}}(\tilde{P}X)$ .
4. Output the span of the top singular vectors of  $\hat{\tilde{P}}$ .

1.  $\|\mathfrak{A}_{\mathcal{B}} - \mathfrak{A}_{\tilde{\mathcal{B}}}\|$  depends on  $\|\mathcal{B} - \tilde{\mathcal{B}}\|$ .
2.  $\|P - \tilde{P}\|_F \leq \text{dist}\left(\text{Adj}(\mathcal{B}), \widetilde{\text{Adj}(\mathcal{B})}\right) \leq \mathcal{O}\left(\|\mathfrak{A}_{\mathcal{B}} - \mathfrak{A}_{\tilde{\mathcal{B}}}\|, \frac{1}{\sigma_{-(s+1)}(\mathfrak{A}_{\mathcal{B}})}\right)$
3.  $\|P_j - \tilde{P}_j\|_F \leq \mathcal{O}\left(\|P - \tilde{P}\|_F, \kappa(M), \frac{1}{\text{eigengap}(P)}\right)$ .
4.  $\text{dist}(U_j, \tilde{U}_j) \leq \mathcal{O}\left(\|P_j - \tilde{P}_j\|_F, \frac{1}{\sigma_{d_j}(P_j)}\right)$ .

Finally, we get

$$\text{dist}(U_j, \tilde{U}_j) \leq \text{poly} \left( \left\| \mathcal{B} - \tilde{\mathcal{B}} \right\|, \kappa(M), \text{eigengap}(P), \frac{1}{\sigma_{d_j}(P_j)}, \frac{1}{\sigma_{-(s+1)}(\mathfrak{A}_{\mathcal{B}})} \right)$$

# Learning Arithmetic Formulas

# Problem

Given a multivariate polynomial  $\tilde{f}(\mathbf{x})$  such that

$$\tilde{f}(\mathbf{x}) \approx T_1(\mathbf{x}) + \cdots + T_s(\mathbf{x})$$

where  $T_i$ 's are *simple* (i.e. low circuit complexity) polynomials, efficiently recover the  $T_i$ 's (approximately).

# Lower Bound Problem

$$f(\mathbf{x}) = T_1(\mathbf{x}) + \cdots + T_s(\mathbf{x})$$

Find an explicit polynomial  $f(\mathbf{x})$  for which  $s$  is large.



# Lower Bound Problem

$$f(\mathbf{x}) = T_1(\mathbf{x}) + \cdots + T_s(\mathbf{x})$$

Find an explicit polynomial  $f(\mathbf{x})$  for which  $s$  is large.

*Lower Bound Proof:* Construct a set of linear maps  $\mathcal{L} = \{L : \mathbb{F}[\mathbf{x}] \rightarrow W\}$  such that

# Lower Bound Problem

$$f(\mathbf{x}) = T_1(\mathbf{x}) + \cdots + T_s(\mathbf{x})$$

Find an explicit polynomial  $f(\mathbf{x})$  for which  $s$  is large.

*Lower Bound Proof:* Construct a set of linear maps  $\mathcal{L} = \{L : \mathbb{F}[\mathbf{x}] \rightarrow W\}$  such that

- $\dim \langle \mathcal{L} \cdot T \rangle$  is small (say,  $\leq r$ ) for all simple  $T$ .
- $\dim \langle \mathcal{L} \cdot f \rangle$  is large (say,  $\geq R$ ) for some polynomial  $f$ .

# Lower Bound Problem

$$f(\mathbf{x}) = T_1(\mathbf{x}) + \cdots + T_s(\mathbf{x})$$

Find an explicit polynomial  $f(\mathbf{x})$  for which  $s$  is large.

*Lower Bound Proof:* Construct a set of linear maps  $\mathcal{L} = \{L : \mathbb{F}[\mathbf{x}] \rightarrow W\}$  such that

- $\dim \langle \mathcal{L} \cdot T \rangle$  is small (say,  $\leq r$ ) for all simple  $T$ .
- $\dim \langle \mathcal{L} \cdot f \rangle$  is large (say,  $\geq R$ ) for some polynomial  $f$ .

Then, we have

$$s \geq \frac{R}{r}.$$

# Lower Bounds to Learning

Lower bound proofs can be used to design algorithms for learning formulas  
(under certain non-degeneracy conditions).

# Lower Bounds to Learning

Lower bound proofs can be used to design algorithms for learning formulas  
(under certain non-degeneracy conditions).

Idea: Reduce it to (robust) vector space decomposition!

# Reduction to VSD

- Input:  $f(\mathbf{x}) \in \mathbb{R}[\mathbf{x}]^{=d}$ .
- Assumption:  $f(\mathbf{x}) = T_1(\mathbf{x}) + T_2(\mathbf{x}) + \cdots + T_s(\mathbf{x})$ ,  $T_i$ 's simple.
- Output:  $T_1(\mathbf{x}), T_2(\mathbf{x}), \dots, T_s(\mathbf{x})$

# Reduction to VSD

- Input:  $f(\mathbf{x}) \in \mathbb{R}[\mathbf{x}]^{=d}$ .
- Assumption:  $f(\mathbf{x}) = T_1(\mathbf{x}) + T_2(\mathbf{x}) + \cdots + T_s(\mathbf{x})$ ,  $T_i$ 's simple.
- Output:  $T_1(\mathbf{x}), T_2(\mathbf{x}), \dots, T_s(\mathbf{x})$
- Lower Bound Maps: Additionally, we require that we have two sets of maps  $\mathcal{L} \subseteq \text{Lin}(\mathbb{R}[\mathbf{x}]^{=d}, V)$  and  $\mathcal{B} \subseteq \text{Lin}(V, W)$  satisfying the following:

# Reduction to VSD

- Input:  $f(\mathbf{x}) \in \mathbb{R}[\mathbf{x}]^{=d}$ .
- Assumption:  $f(\mathbf{x}) = T_1(\mathbf{x}) + T_2(\mathbf{x}) + \cdots + T_s(\mathbf{x})$ ,  $T_i$ 's simple.
- Output:  $T_1(\mathbf{x}), T_2(\mathbf{x}), \dots, T_s(\mathbf{x})$
- Lower Bound Maps: Additionally, we require that we have two sets of maps  $\mathcal{L} \subseteq \text{Lin}(\mathbb{R}[\mathbf{x}]^{=d}, V)$  and  $\mathcal{B} \subseteq \text{Lin}(V, W)$  satisfying the following:

**Property 1** : Direct Sum Decomposition under  $\mathcal{L}$ :

$$U = \langle \mathcal{L} \cdot f \rangle = \langle \mathcal{L} \cdot T_1 \rangle \oplus \langle \mathcal{L} \cdot T_2 \rangle \oplus \cdots \oplus \langle \mathcal{L} \cdot T_s \rangle = U_1 \oplus \cdots \oplus U_s.$$



# Reduction to VSD

- Input:  $f(\mathbf{x}) \in \mathbb{R}[\mathbf{x}]^{=d}$ .
- Assumption:  $f(\mathbf{x}) = T_1(\mathbf{x}) + T_2(\mathbf{x}) + \cdots + T_s(\mathbf{x})$ ,  $T_i$ 's simple.
- Output:  $T_1(\mathbf{x}), T_2(\mathbf{x}), \dots, T_s(\mathbf{x})$
- Lower Bound Maps: Additionally, we require that we have two sets of maps  $\mathcal{L} \subseteq \text{Lin}(\mathbb{R}[\mathbf{x}]^{=d}, V)$  and  $\mathcal{B} \subseteq \text{Lin}(V, W)$  satisfying the following:

**Property 1** : Direct Sum Decomposition under  $\mathcal{L}$ :

$$U = \langle \mathcal{L} \cdot f \rangle = \langle \mathcal{L} \cdot T_1 \rangle \oplus \langle \mathcal{L} \cdot T_2 \rangle \oplus \cdots \oplus \langle \mathcal{L} \cdot T_s \rangle = U_1 \oplus \cdots \oplus U_s.$$

**Property 2** : Direct Sum Decomposition under  $\mathcal{B}$ :

$$V = \mathcal{B} \cdot U = \langle \mathcal{B} \cdot \mathcal{L} \cdot T_1 \rangle \oplus \langle \mathcal{B} \cdot \mathcal{L} \cdot T_2 \rangle \oplus \cdots \oplus \langle \mathcal{B} \cdot \mathcal{L} \cdot T_s \rangle = V_1 \oplus \cdots \oplus V_s.$$

# Reduction to VSD

$(U, V, \mathcal{B})$  is an instance of vector space decomposition!

# Reduction to VSD

$(U, V, \mathcal{B})$  is an instance of vector space decomposition!

In the noisy setting, we are given  $\tilde{f}(\mathbf{x}) = f(\mathbf{x}) + \eta(\mathbf{x})$  for some noise  $\eta(\mathbf{x})$ .

We approximate

$\tilde{U}$  as the  $\dim(U)$ -dimensional approximation to  $\langle \mathcal{L} \cdot \tilde{f} \rangle$ , and similarly for  $\tilde{V}$ .

# Reduction to VSD

$(U, V, \mathcal{B})$  is an instance of vector space decomposition!

In the noisy setting, we are given  $\tilde{f}(\mathbf{x}) = f(\mathbf{x}) + \eta(\mathbf{x})$  for some noise  $\eta(\mathbf{x})$ .

We approximate

$\tilde{U}$  as the  $\dim(U)$ -dimensional approximation to  $\langle \mathcal{L} \cdot \tilde{f} \rangle$ , and similarly for  $\tilde{V}$ .

$(\tilde{U}, \tilde{V}, \mathcal{B})$  is an instance of robust vector space decomposition!

# Recovery Guarantees

## Theorem (Informal)

*Under the appropriate assumptions, there is an efficient algorithm which recovers  $\tilde{T}_1, \dots, \tilde{T}_s$  such that, with probability  $1 - \delta$ ,*

$$\left\| T_i - \tilde{T}_i \right\| \leq \text{poly}(s, d, d_U, d_V, 1/\delta, 1/\sigma_1, 1/\sigma_2) \cdot \epsilon.$$

*where*

- 1.  $d_U = \dim(U)$ ,  $d_V = \dim(V)$ .*
- 2.  $\sigma_1$  is the smallest non-zero singular value of a matrix associated to  $U$ .*
- 3.  $\sigma_2$  is the smallest non-zero singular value of the appropriate adjoint operator.*

# Recovery Guarantees

## Theorem (Informal)

*Under the appropriate assumptions, there is an efficient algorithm which recovers  $\tilde{T}_1, \dots, \tilde{T}_s$  such that, with probability  $1 - \delta$ ,*

$$\left\| T_i - \tilde{T}_i \right\| \leq \text{poly}(s, d, d_U, d_V, 1/\delta, 1/\sigma_1, 1/\sigma_2) \cdot \epsilon.$$

*where*

1.  $d_U = \dim(U), d_V = \dim(V)$ .
2.  $\sigma_1$  is the smallest non-zero singular value of a matrix associated to  $U$ .
3.  $\sigma_2$  is the smallest non-zero singular value of the appropriate adjoint operator.

Note that the bounds here are very pessimistic for random  $\eta$ .

# Applications: Unsupervised Learning

# Unsupervised Learning

A general problem template:

Given samples  $S \subset \mathbb{R}^n$  from a well-defined distribution  $D$ , recover the parameters of the distribution.



# Unsupervised Learning

A general problem template:

Given samples  $S \subset \mathbb{R}^n$  from a well-defined distribution  $D$ , recover the parameters of the distribution.

- Mixtures of Gaussians

# Unsupervised Learning

A general problem template:

Given samples  $S \subset \mathbb{R}^n$  from a well-defined distribution  $D$ , recover the parameters of the distribution.

- Mixtures of Gaussians
- Subspace Clustering

# Unsupervised Learning

A general problem template:

Given samples  $S \subset \mathbb{R}^n$  from a well-defined distribution  $D$ , recover the parameters of the distribution.

- Mixtures of Gaussians
- Subspace Clustering
- Topic Modeling
- ...

# Solution Template

Method of moments approach: estimate the moments of the distribution, and from the moments, recover the parameters.

# Solution Template

Method of moments approach: estimate the moments of the distribution, and from the moments, recover the parameters.

Putting the (appropriate combinations of) moments as coefficients of a polynomial  $f(\mathbf{x})$ , we often have that  $f$  has a small formula.

# Solution Template

Method of moments approach: estimate the moments of the distribution, and from the moments, recover the parameters.

Putting the (appropriate combinations of) moments as coefficients of a polynomial  $f(\mathbf{x})$ , we often have that  $f$  has a small formula.

Learning the small formula for  $f$  implies learning the parameters of the distribution!

## Example - Mixtures of Gaussians

Samples  $S \subset \mathbb{R}^n$  generated by a mixture of  $s$  zero-mean Gaussians.

# Example - Mixtures of Gaussians

Samples  $S \subset \mathbb{R}^n$  generated by a mixture of  $s$  zero-mean Gaussians.  
 $d$ -th order moment polynomial for this distribution:

$$p(\mathbf{x}) = q_1^{d/2}(\mathbf{x}) + q_2^{d/2}(\mathbf{x}) + \cdots + q_s^{d/2}(\mathbf{x}),$$

where  $q_i$ 's are quadratic polynomials encoding the covariance matrices of the Gaussians.



# Example - Mixtures of Gaussians

Samples  $S \subset \mathbb{R}^n$  generated by a mixture of  $s$  zero-mean Gaussians.  
 $d$ -th order moment polynomial for this distribution:

$$p(\mathbf{x}) = q_1^{d/2}(\mathbf{x}) + q_2^{d/2}(\mathbf{x}) + \cdots + q_s^{d/2}(\mathbf{x}),$$

where  $q_i$ 's are quadratic polynomials encoding the covariance matrices of the Gaussians.

Following [GKS20], we can use  $\mathcal{L} = \sigma_{x_{m+1}=\dots=x_n=0} \cdot \partial_{\mathbf{x}}^k$  and

$\mathcal{B} = \mathbf{y}^1 \cdot \partial_y^1$  to reduce the problem to robust VSD.

## Example - Subspace Clustering

Samples  $A \subset \mathbb{R}^n$  are such that there exist low-dimensional, disjoint subspaces  $S_1, \dots, S_s$  such that each point lies (approximately) in one of the  $S_i$ 's.

# Example - Subspace Clustering

Samples  $A \subset \mathbb{R}^n$  are such that there exist low-dimensional, disjoint subspaces  $S_1, \dots, S_s$  such that each point lies (approximately) in one of the  $S_i$ 's.

Let  $U = \langle A^{\otimes d} \rangle$ ,  $V = \langle A^{\otimes d-1} \rangle$ . Then,  $U = S_1^{\otimes d} \oplus \dots S_s^{\otimes d}$ ,

$$V = S_1^{\otimes d-1} \oplus \dots S_s^{\otimes d-1}.$$

# Example - Subspace Clustering

Samples  $A \subset \mathbb{R}^n$  are such that there exist low-dimensional, disjoint subspaces  $S_1, \dots, S_s$  such that each point lies (approximately) in one of the  $S_i$ 's.

Let  $U = \langle A^{\otimes d} \rangle$ ,  $V = \langle A^{\otimes d-1} \rangle$ . Then,  $U = S_1^{\otimes d} \oplus \dots S_s^{\otimes d}$ ,

$V = S_1^{\otimes d-1} \oplus \dots S_s^{\otimes d-1}$ . Let  $\mathcal{B} = \partial^{\otimes d-1}$ . Then, solving the robust VSD

problem on  $(U, V, \mathcal{B})$  allows us to recover  $S_i$ 's.

# Smoothed Analysis Conjectures

The error bounds mainly depend on two singular values - the smallest non-zero singular value of the adjoint algebra associated to the problem, and the smallest non-zero singular value of a matrix associated to  $U$ .

# Smoothed Analysis Conjectures

The error bounds mainly depend on two singular values - the smallest non-zero singular value of the adjoint algebra associated to the problem, and the smallest non-zero singular value of a matrix associated to  $U$ .

## conjecture (Informal)

*The minimum singular values associated with the adjoint algebra and the  $U$ -associated matrix are 'not too small' in perturbed instances.*

# Smoothed Analysis Conjectures

The error bounds mainly depend on two singular values - the smallest non-zero singular value of the adjoint algebra associated to the problem, and the smallest non-zero singular value of a matrix associated to  $U$ .

## conjecture (Informal)

*The minimum singular values associated with the adjoint algebra and the  $U$ -associated matrix are 'not too small' in perturbed instances.*

Conjecture implies an efficient algorithm for mixtures of Gaussians, subspace clustering, and other unsupervised learning problems in the smoothed setting.

# Smoothed Analysis Conjectures

The error bounds mainly depend on two singular values - the smallest non-zero singular value of the adjoint algebra associated to the problem, and the smallest non-zero singular value of a matrix associated to  $U$ .

## conjecture (Informal)

*The minimum singular values associated with the adjoint algebra and the  $U$ -associated matrix are 'not too small' in perturbed instances.*

Conjecture implies an efficient algorithm for mixtures of Gaussians, subspace clustering, and other unsupervised learning problems in the smoothed setting.

As a first step we prove the conjecture for the singular value of the adjoint algebra for subspace clustering.



# References

1. Rong Ge and Qingqing Huang and Sham M. Kakade. *Learning Mixtures of Gaussians in High Dimensions*. in *Proceedings of the Forty-Seventh Annual ACM on Symposium on Theory of Computing, STOC (2015)*, 761–770

# Thank You